

NoFluxo UNB

Project Documentation

Squad 03 — MDS 2025/1
(Métodos de Desenvolvimento de Software)
FGA/UnB

Live URL: <https://no-fluxo.com>

May 22, 2026

Contents

1	Overview	3
2	Project Structure	3
3	Backend	4
3.1	Directory Structure	4
3.2	Key Dependencies	4
3.3	Controllers	5
3.4	Core Business Logic: Discipline Matching	5
3.5	PDF Parsing (Python)	5
3.6	Logging	5
4	Frontend	6
4.1	Directory Structure	6
4.2	Pages & Routes	7
4.3	Key Component Groups	7
4.4	State Management	8
4.5	Services Layer	8
4.6	Key Data Flows	9
4.7	Path Aliases	9
5	Database	9
5.1	Tables	9
5.2	Key Relationships	10
5.3	Key Fields	10
5.4	Views	10
5.5	Custom Functions	10
5.6	Row Level Security (RLS)	10
6	Data Collection	11
6.1	Scraping Scripts	11
6.2	Data Files	11
6.3	Pipeline	11
7	Infrastructure & Deployment	11
7.1	Kubernetes (Production)	11
7.2	Deploy Scripts	12
7.3	Dockerfiles	12
7.4	Reference Documentation	12
7.5	CORS Configuration	12
8	CI/CD & Testing	13
8.1	GitHub Actions Workflows	13
8.2	Testing Tools	13
8.3	Quality Tools	13
9	API Reference	13

10 Authentication & Security	15
10.1 Authentication Methods	15
10.2 Auth Flow	15
10.3 Route Protection	15
10.4 Backend Authorization	15
10.5 Row Level Security	15
11 AI Assistant Integration	16
11.1 RAGFlow	16
11.2 Flow	16
11.3 Response Format	16
11.4 Data Pipeline for AI	16

1 Overview

NoFluxo is a full-stack web application that helps students at the Universidade de Brasília (UnB) visualize and plan their academic curriculum. Students can upload their academic transcript (PDF), and the system parses it, matches completed disciplines to the official curriculum, and displays an interactive flowchart showing their progress.

Live URL: <https://no-fluxo.com>

Core capabilities:

- Interactive curriculum flowchart with prerequisite visualization
- PDF transcript parsing (browser-side + OCR fallback)
- Discipline matching algorithm (transcript vs. official curriculum)
- Equivalency detection across curriculum versions
- AI-powered discipline recommendations (via RAGFlow)
- Google OAuth, email/password, and anonymous authentication
- Progress tracking with mandatory/elective hour calculations

Tech Stack:

Layer	Technology
Frontend	SvelteKit 2 + Svelte 5, Tailwind CSS 4, Vite 7
Backend	Node.js + TypeScript + Express.js
PDF Processing	Python (PyMuPDF + Tesseract OCR)
Database	Supabase (PostgreSQL)
AI	RAGFlow agent
Deployment	Kubernetes (K3s), Docker, GitHub Actions
CI/CD	GitHub Actions

Team: Squad 03 — MDS 2025/1 (Métodos de Desenvolvimento de Software), FGA/UnB.

2 Project Structure

```

2025-1-NoFluxoUNB/
|-- no_fluxo_backend/           # TypeScript/Express backend + Python PDF parsing
|-- no_fluxo_frontend_svelte/  # SvelteKit frontend application
|-- coleta_dados/              # Data collection/scraping scripts (Python)
|-- documentacao/              # Project documentation (Portuguese)
|-- kubernetes_docs/           # Kubernetes deployment docs & templates
|-- plans/                     # Planning and strategy documents
|-- test_historicos/           # Test PDF transcripts for parsing
|-- tests-python/              # Python test suite
|-- scripts/
|   '-- deploy/                 # Deploy scripts for Kubernetes (deploy_local.py)
|-- .github/                   # GitHub Actions CI/CD workflows
|-- k8s.backend.Dockerfile      # Kubernetes backend image
|-- k8s.frontend-svelte.Dockerfile # Kubernetes frontend image
|-- requirements.txt            # Root Python dependencies
|-- README.md                  # Main project README
|-- CONTRIBUTING.md            # Contribution guidelines
|-- COMMIT_GUIDELINES.md       # Git commit conventions
|-- CODE_OF_CONDUCT.md         # Code of conduct
|-- SECURITY.md                 # Security policy
'-- LICENSE                    # GNU GPLv3 License

```

3 Backend

Location: no_fluxo_backend/

Port: 3325

Language: TypeScript (Express.js) + Python (PDF parsing)

3.1 Directory Structure

```
no_fluxo_backend/
|-- src/
|   |-- index.ts                # Express server entry point
|   |-- interfaces.ts           # TypeScript interfaces
|   |-- supabase_wrapper.ts     # Supabase client wrapper
|   |-- logger.ts               # Winston logger config
|   |-- utils.ts                # General utilities
|   |-- controllers/
|       |-- fluxograma_controller.ts # Core flowchart/matching logic
|       |-- users_controller.ts     # User registration
|       |-- cursos_controller.ts    # Course listing
|       |-- materias_controller.ts  # Discipline lookup
|       |-- assistente_controller.ts # AI assistant proxy
|       |-- testes_controller.ts    # Debug/test endpoints
|   |-- services/
|       |-- ragflow.service.ts      # RAGFlow API client
|       |-- ragflow.types.ts        # RAGFlow type definitions
|   |-- utils/
|       |-- controller_logger.ts    # Scoped controller logging
|       |-- text.utils.ts           # Text processing (accents, HTML)
|       |-- ranking.formatter.ts    # AI response -> Markdown formatter
|-- parse-pdf/
|   |-- pdf_parser_final.py        # Main PDF parser (PyMuPDF + Tesseract)
|   |-- pdf_parser_ocr.py          # OCR-only parser
|   |-- requirements.txt           # Python dependencies
|   |-- tests/                     # Python tests
|-- scripts/
|   |-- export_schema.ts           # Database schema export tool
|   |-- setup-tesseract.js         # Tesseract OCR setup
|   |-- migrations/                # SQL migration files
|-- docs/
|   |-- database_schema.json       # Exported full schema
|   |-- database_tables.md         # Table documentation
|   |-- database_functions.sql     # Stored functions
|   |-- rls_policies.sql           # Row-level security policies
|   |-- supabase_migrations/       # Migration SQL files
|-- package.json
|-- tsconfig.json
'-- Dockerfile
```

3.2 Key Dependencies

- **express** — HTTP framework
- **@supabase/supabase-js** — Supabase client
- **axios** — HTTP client (RAGFlow calls)
- **helmet** — Security headers
- **cors** — Cross-origin middleware
- **express-fileupload** — File upload handling

- **express-ws** — WebSocket support
- **winston / morgan** — Logging

3.3 Controllers

Controller	Base Route	Purpose
<code>fluxograma_controller.ts</code>	<code>/fluxograma</code>	Flowchart retrieval, discipline matching, save/delete
<code>users_controller.ts</code>	<code>/users</code>	User registration (Google OAuth, email)
<code>cursos_controller.ts</code>	<code>/cursos</code>	Course catalog listing
<code>materias_controller.ts</code>	<code>/materias</code>	Discipline lookup by code/name
<code>assistente_controller.ts</code>	<code>/assistente</code>	AI assistant proxy to RAGFlow
<code>testes_controller.ts</code>	<code>/testes</code>	Debug/testing endpoints

3.4 Core Business Logic: Discipline Matching

The most important endpoint is `POST /fluxograma/casar_disciplinas`. It runs a **4-stage matching algorithm**:

1. **Primary Matrix Matching** — Matches extracted disciplines against the student's curriculum version by code and name.
2. **Alternative Matrix Matching** — Checks other curriculum versions of the same course.
3. **Equivalency Matching** — Searches the equivalency table for matching logical expressions.
4. **Classification** — Categorizes each discipline by status (`APR`, `CUMP`, `MATR`, `REP`) and type (`obrigatoria`, `optativa`).

The algorithm handles duplicates (same discipline with different statuses), uses priority-based status selection, and computes completion metrics (hours, percentages, IRA).

3.5 PDF Parsing (Python)

Located in `parse-pdf/`, the Python service extracts structured data from UnB academic transcript PDFs:

- Uses **PyMuPDF (fitz)** for text extraction
- Falls back to **Tesseract OCR** for scanned documents
- Extracts: course name, curriculum version, disciplines (code, name, status, grade, hours), IRA, weighted average, frequency
- Handles SIGAA (UnB's academic system) format

3.6 Logging

Uses Winston with three transports:

- Console (colored output)
- `logs/error.log` (errors only)
- `logs/all.log` (all levels)

Controller-level scoped logging via `ControllerLogger` class: `[Controller] [Endpoint] message`.

4 Frontend

Location: no_fluxo_frontend_svelte/

Framework: SvelteKit 2 + Svelte 5 (runes)

Styling: Tailwind CSS 4

Build Tool: Vite 7

4.1 Directory Structure

```
no_fluxo_frontend_svelte/src/
|-- routes/                                # SvelteKit pages
|   |-- +layout.svelte                    # Root layout (auth + navigation)
|   |-- +page.svelte                     # Landing page
|   |-- login/                           # Login page
|   |-- signup/                           # Registration page
|   |-- login-anonimo/                   # Anonymous login
|   |-- password-recovery/               # Password recovery
|   |-- auth/callback/                   # OAuth callback
|   |-- auth/reset-password/             # Password reset
|   |-- fluxogramas/                     # Course listing page
|   |-- meu-fluxograma/                   # User's flowchart
|   |-- meu-fluxograma/[courseName]/     # Dynamic course flowchart
|   |-- upload-historico/                 # PDF transcript upload
|   |-- assistente/                       # AI assistant (in development)
|   |-- api/casar-disciplinas/           # SvelteKit API route
|   |-- health/                           # Health check
|   |-- sitemap.xml/                     # SEO sitemap
|
|-- lib/
|   |-- components/
|   |   |-- auth/                         # Login, Signup, Logout, Recovery forms
|   |   |-- fluxograma/                   # Flowchart display components
|   |   |-- upload/                       # PDF upload flow components
|   |   |-- layout/                       # Navbar, Sidebar, Footer, Logo, etc.
|   |   |-- home/                         # Landing page sections
|   |   |-- effects/                      # AnimatedBackground, GraffitiBackground
|   |   |-- forms/                        # Select, TextInput
|   |   |-- ui/                           # bits-ui wrappers (Button, Card, Dialog, etc.)
|   |   |-- icons/                        # GoogleIcon
|   |   |-- seo/                          # PageMeta, JsonLd
|   |
|   |-- services/
|   |   |-- auth.service.ts               # Supabase authentication
|   |   |-- fluxograma.service.ts         # Flowchart data loading
|   |   |-- upload.service.ts             # PDF upload & processing
|   |   |-- assistente.service.ts        # AI assistant API
|   |   |-- integralizacao.service.ts     # Completion calculation
|   |   |-- user.service.ts               # User profile operations
|   |   |-- supabase-data.service.ts     # Direct Supabase queries (RLS)
|   |   |-- pdf/
|   |   |   |-- pdfParser.ts              # PDF parsing orchestrator
|   |   |   |-- pdfExtractor.ts           # Text extraction (PDF.js)
|   |   |   |-- pdfDataExtractor.ts       # Structured data parsing
|   |   |   |-- pdfPositionExtractor.ts   # Position-based parsing
|   |
|   |-- stores/                           # State management
|   |   |-- auth.ts                       # Auth state (Svelte 4 writable)
```

```

| | |-- fluxograma.store.svelte.ts # Flowchart state (Svelte 5 runes)
| | |-- chatStore.ts             # AI chat state
| | |-- uploadStore.ts          # Upload progress state
| | |-- modal.ts                # Modal visibility
| | |-- navigation.ts           # Navigation state
| | |-- theme.ts                 # Dark mode
| |
| |-- types/                     # TypeScript type definitions
| |-- schemas/                  # Zod validation schemas
| |-- guards/                   # Route protection (authGuard.ts)
| |-- hooks/                    # useBreakpoint, useMediaQuery, useSession
| |-- supabase/                 # Browser Supabase client
| |-- server/                   # Server Supabase client
| |-- utils/                    # Helpers (API, breakpoints, expressions, toast)
| |-- actions/                  # Svelte actions (clickOutside, inview)
| |-- factories/                 # Data transformation factories
| |-- config.ts                 # Environment config
|
|-- app.css                     # Global Tailwind styles
|-- app.d.ts                    # Global type declarations
'-- hooks.server.ts             # Server hooks (auth, redirects)

```

4.2 Pages & Routes

Route	Auth Required	Purpose
/	No	Landing page (hero, features, about)
/login	No	Email/password login
/signup	No	User registration
/password-recovery	No	Password reset request
/login-anonimo	No	Anonymous browsing mode
/auth/callback	No	Google OAuth callback handler
/auth/reset-password	No	Set new password form
/fluxogramas	Yes	List all available courses/flowcharts
/meu-fluxograma	Yes	User's personal flowchart overview
/meu-fluxograma/[course]	Yes	Course-specific interactive flowchart
/upload-historico	Yes	Upload PDF academic transcript
/assistente	Yes	AI assistant (in development)

4.3 Key Component Groups

Flowchart Components — The main feature of the app:

- `FluxogramContainer.svelte` — Interactive grid with pan/zoom (mouse + touch pinch)
- `SemesterColumn.svelte` — Renders a single semester's subjects in a column
- `SubjectCard.svelte` — Individual subject card (color-coded by status)
- `PrerequisiteConnections.svelte` — SVG lines connecting prerequisites
- `SubjectDetailsModal.svelte` — Detailed subject info on click
- `PrerequisiteChainDialog.svelte` — Full prerequisite chain visualization

- `OptativasModal.svelte` — Elective subject selection
- `ProgressSummarySection.svelte` — Completion summary (mandatory/elective hours)
- `IntegralizacaoSection.svelte` — Credit requirements progress

Upload Components — PDF processing flow:

- `FileDropzone.svelte` — Drag-and-drop PDF upload area
- `UploadProgress.svelte` — Progress indicator
- `CourseSelectionModal.svelte` — Course disambiguation
- `DisciplinaList.svelte` — Matched discipline listing
- `UploadSuccess.svelte` — Success screen
- `ProcessingResults.svelte` — Parsing results

Layout Components:

- `Navbar.svelte`, `Sidebar.svelte`, `Footer.svelte`
- `AppLogo.svelte`, `Breadcrumbs.svelte`
- `LoadingBar.svelte`, `SplashScreen.svelte`
- `GlassContainer.svelte` — Glassmorphism container
- `GradientCTAButton.svelte` — Animated CTA button

Landing Page Sections:

- `HeroSection.svelte` — Main banner
- `ComoFuncionaSection.svelte` — “How it works” steps
- `ProntoParaOrganizarSection.svelte` — Call to action
- `SobreNosSection.svelte` — About the team

4.4 State Management

The app uses a mix of Svelte 4 writable stores and Svelte 5 runes:

Store	Type	Purpose
<code>auth.ts</code>	Svelte writable	Auth state, user data, localStorage sync
<code>fluxograma.store.svelte.ts</code>	Svelte 5 runes	Course data, zoom, selected subjects
<code>uploadStore.ts</code>	Svelte writable	Upload progress, matched disciplines
<code>chatStore.ts</code>	Svelte writable	AI chat messages
<code>modal.ts</code>	Svelte writable	Modal open/close state
<code>navigation.ts</code>	Svelte writable	Navigation state
<code>theme.ts</code>	Svelte writable	Dark/light mode

4.5 Services Layer

Service	Purpose
<code>auth.service.ts</code>	Supabase Auth (login, signup, OAuth, anonymous, password)
<code>fluxograma.service.ts</code>	Load course data and build flowchart models

Service	Purpose
upload.service.ts	Browser-side PDF parsing + discipline matching
supabase-data.service.ts	Direct Supabase queries with RLS (courses, matrices, subjects)
integralizacao.service.ts	Calculate mandatory/elective hour completion
assistente.service.ts	Proxy to AI assistant backend
pdf/pdfParser.ts	PDF.js-based browser-side transcript parsing

4.6 Key Data Flows

PDF Upload Flow:

```
FileDropzone -> uploadService.parsePdfLocally() -> PDF.js parsing (browser)
-> Extract disciplines -> POST /api/casar-disciplinas -> Backend matching
-> Display results -> User confirms -> saveFluxogramaData() -> Navigate to flowchart
```

Flowchart View Flow:

```
Route loads -> authStore user data -> fluxogramaService.getCourseData()
-> Query Supabase (RLS) -> Build CursoModel -> fluxogramaStore
-> Render FluxogramContainer -> Semester columns + subject cards + SVG connections
```

4.7 Path Aliases

```
$components -> src/lib/components
$lib         -> src/lib
$stores      -> src/lib/stores
$types       -> src/lib/types
$services    -> src/lib/services
$utils       -> src/lib/utils
```

5 Database

Platform: Supabase (PostgreSQL)

URL: <https://lijmhbstgdinsukovyfl.supabase.co>

5.1 Tables

Table	Rows	Size	Purpose
users	162	136 KB	User accounts (linked to Supabase auth)
dados_users	99	600 KB	User flowchart data and current semester
cursos	102	64 KB	UnB courses (Administração, Engenharia, etc.)
matrizes	235	96 KB	Curriculum versions per course

Table	Rows	Size	Purpose
materias	25,624	8.4 MB	Discipline catalog (code, name, hours, syllabus)
materias_por_curso	65,955	13.1 MB	Discipline-to-curriculum mapping (with level)
pre_requisitos	14,573	2.3 MB	Prerequisite relationships
co_requisitos	185	80 KB	Corequisite relationships
equivalencias	19,445	4 MB	Subject equivalencies across curricula

5.2 Key Relationships

```

auth.users (Supabase Auth)
  '-- users.auth_id
    '-- dados_users.id_user      (user's saved flowchart)

cursos
  '-- matrizes.id_curso          (curriculum versions)
    '-- materias_por_curso.id_matriz
      '-- materias.id_materia (disciplines in curriculum)

materias
  |-- pre_requisitos (self-referential: materia -> materia_requisito)
  |-- co_requisitos  (self-referential: materia -> materia_corequisito)
  '-- equivalencias  (materia + curso + matriz)

```

5.3 Key Fields

- `materias_por_curso.nivel` — 0 = elective, 1-8 = mandatory at that semester
- `equivalencias.expressao_logica` — JSONB with logical expressions for equivalency rules
- `matrizes.curriculo_completo` — Unique curriculum identifier string
- `matrizes.ch_obrigatoria_exigida` / `ch_optativa_exigida` — Required hours for graduation

5.4 Views

- `vw_creditos_por_matriz` — Joins `matrizes` → `cursos` → `materias_por_curso` → `materias` to compute credit totals per curriculum version.

5.5 Custom Functions

- `export_schema()` — Exports complete database schema as JSONB
- `calcular_creditos_por_curso(id_curso)` — Calculates credits (1 credit = 15 hours)
- `atualizar_creditos_cursos()` — Batch update credits for all courses

5.6 Row Level Security (RLS)

All tables have RLS enabled. Policies:

- **Academic data** (`cursos`, `materias`, `matrizes`, `pre_requisitos`, `co_requisitos`, `equivalencias`): **Public read** for authenticated users.
- **users**: Users can only read/update their **own** record (`WHERE auth_id = auth.uid()`).

- **dados__users**: Full CRUD restricted to the **owning user** only.

6 Data Collection

Location: coleta_dados/

6.1 Scraping Scripts

Python scripts that scrape data from UnB's SIGAA (academic information system):

Script	Purpose
scraping_cursos.py	Scrape course information
scraping_atual.py	Current course data
scraping_ementa.py	Course syllabi
scraping_equivalencias.py	Subject equivalencies
scraping_sigaa.py	SIGAA system integration
extrair_turmas_sigaa.py	Extract class sections
integracao_banco.py (51 KB)	Main database integration layer
insere_departamento.py	Insert department data
insere_tipocurso.py	Insert course type data
confere_estruturas.py	Verify data structures
formatar_para_ragflow.py	Format data for RAGFlow AI

6.2 Data Files

- dados/cursos-de-graduacao.json — 102+ UnB courses with metadata
- dados/materias/turmas_depto_*.json — Class sections organized by department (20+ files)

6.3 Pipeline

```
SIGAA website -> Scraping scripts -> JSON data files -> integracao_banco.py ->
Supabase DB
RAGFlow AI
-> formatar_para_ragflow.py ->
```

7 Infrastructure & Deployment

7.1 Kubernetes (Production)

Production runs on a K3s cluster. The deploy workflow is:

```
Developer runs scripts/deploy/deploy_local.py backend|frontend
-> Docker build (multi-stage, local or buildx)
-> Push to registry.kubernetes.crianex.com
-> Deploy API resolves image digest
-> Deploy API creates K8s manifests and applies via kubectl
-> Service available at crianex.com domain
```

Cluster details:

- **Platform:** K3s cluster at `kubernetes.crianex.com`
- **Ingress:** Traefik (handles SSL/TLS via cert-manager + Let's Encrypt)
- **Registry:** Private Docker registry (`registry.kubernetes.crianex.com`)
- **Deploy API:** REST API at `deploy.kubernetes.crianex.com`
- **Monitoring:** Prometheus + Grafana + Loki

Production domains:

- Frontend: `https://no-fluxo.crianex.com`
- Backend API: `https://api-nofluxo.crianex.com`

7.2 Deploy Scripts

Located in `scripts/deploy/`:

File	Purpose
<code>deploy_local.py</code>	Main deploy script. Builds Docker images, pushes to registry, deploys via API
<code>deploy_config.py</code>	App configuration (names, ports, domains, env vars) for backend and frontend
<code>requirements.txt</code>	Python dependencies (<code>requests</code> , <code>python-dotenv</code>)
<code>.deploy/</code>	Cached build IDs to avoid redundant rebuilds

7.3 Dockerfiles

File	Target	Base	Description
<code>k8s.backend.Dockerfile</code>	Backend	Node 20 Alpine	Multi-stage build, non-root user, health check, runs <code>node dist/index.js</code>
<code>k8s.frontend-svelte.Dockerfile</code>	Frontend	Node 20 Bullseye	Multi-stage build, pnpm, adapter-node for SSR, build-time env vars

7.4 Reference Documentation

The `kubernetes_docs/` folder contains reference documentation and templates for the Kubernetes cluster setup, including registry auth guides, monitoring dashboards, and generic Dockerfile templates.

7.5 CORS Configuration

Allowed origins:

- `https://www.no-fluxo.com`
- `https://no-fluxo.com`
- `http://localhost:3000`
- `http://localhost:5000`

8 CI/CD & Testing

8.1 GitHub Actions Workflows

Workflow	Purpose
all-tests.yml	Run TypeScript + Python tests
typescript-tests.yml	ESLint + type-check + Jest
python-tests.yml	pytest on Python 3.9, 3.10, 3.11
security-and-quality.yml	Security scans + code quality checks
pipelineCI.yml	CI pipeline

8.2 Testing Tools

Tool	Language	Purpose
Jest	TypeScript	Backend unit tests
Vitest	TypeScript	Frontend unit tests
Playwright	TypeScript	E2E integration
pytest	Python	PDF parser tests

8.3 Quality Tools

- **TypeScript:** ESLint, Prettier, TypeScript strict mode
- **Python:** flake8, black, isort, mypy, bandit, safety
- **Coverage:** Codecov integration

9 API Reference

Backend API (Express, port 3325)

Health

Method	Endpoint	Description
GET	/	Server info + endpoint listing
GET	/health	Kubernetes health probe

Users (/users)

Method	Endpoint	Body	Description
POST	/users/register-user-with-google	<code>{ email, nome_completo }</code>	Register via Google OAuth
GET	/users/get-user-by-email	Query: email	Get user profile

Method	Endpoint	Body	Description
POST	/users/registrar-user-with-email,	{ email, nome_completo }	Register with email

Courses (/cursos)

Method	Endpoint	Description
GET	/cursos/all-cursos	List all courses with credit info

Disciplines (/materias)

Method	Endpoint	Body	Description
GET	/materias/materias-name-by-codes	{ codes: string[] }	Get names by codes
POST	/materias/materias-from-codigos	{ codigos: string[], id_curso }	Get detailed subject info

Flowchart (/fluxograma)

Method	Endpoint	Body / Query	Description
GET	/fluxograma/fluxograma	Query: nome_curso	Get full curriculum flowchart
POST	/fluxograma/casar_disciplina	Extracted PDF data	Match transcript to curriculum
POST	/fluxograma/upload-dados-fluxograma,	{ fluxograma, periodo_letivo }	Save flowchart to user profile
DELETE	/fluxograma/delete-fluxograma	Headers: Authorization, User-ID	Remove user's saved flowchart

AI Assistant (/assistente)

Method	Endpoint	Body	Description
POST	/assistente/analyze	{ materia: string }	Get AI discipline recommendations

Method	Endpoint	Body	Description
GET	/assistente/health	—	Check AI service status

Frontend API Routes (SvelteKit)

Method	Endpoint	Description
POST	/api/casar-disciplinas	Proxies discipline matching
GET	/health	Frontend health check
GET	/sitemap.xml	SEO sitemap

10 Authentication & Security

10.1 Authentication Methods

1. **Email/Password** — Standard Supabase Auth signup and login
2. **Google OAuth** — Single sign-on via Google
3. **Anonymous Mode** — Browse flowcharts without an account (limited features)

10.2 Auth Flow

```
Client (LoginForm) -> Supabase Auth (signIn/signUp/signInWithOAuth)
-> Receive session token -> authService.databaseSearchUser()
-> Query users table -> Set authStore -> Navigate to protected route
```

10.3 Route Protection

- **Server-side:** `hooks.server.ts` validates the Supabase session on every request to protected routes. Invalid sessions redirect to `/login?redirect=<path>`.
- **Client-side:** `authGuard.ts` checks auth state before navigation.

10.4 Backend Authorization

Protected backend endpoints (`upload-dados-fluxograma`, `delete-fluxograma`) require:

1. Authorization header with Bearer token (Supabase session)
2. User-ID header
3. Token validation against Supabase auth
4. Email match verification between token and database

10.5 Row Level Security

All Supabase tables use RLS policies. Academic data is read-only for authenticated users. User-specific data (`users`, `dados_users`) is restricted to the owning user.

11 AI Assistant Integration

11.1 RAGFlow

The AI assistant uses **RAGFlow**, an external AI agent service, to provide discipline recommendations.

Configuration:

- RAGFLOW_API_KEY — API authentication
- RAGFLOW_BASE_URL — RAGFlow server URL
- RAGFLOW_AGENT_ID — Specific agent ID

11.2 Flow

```
User submits subject name -> Backend normalizes text (remove accents, uppercase)
-> Create RAGFlow session -> Call RAGFlow completions API
-> Parse AI response -> Format as Markdown ranking
-> Return to client
```

11.3 Response Format

The AI returns a ranked list of recommended disciplines with:

- Discipline name and code
- Responsible department
- Relevance score (0–100)
- Justification text
- Syllabus excerpt

11.4 Data Pipeline for AI

Course data is formatted and uploaded to RAGFlow via `coleta_dados/scraping/formatar_para_ragflow.py` which structures syllabus and curriculum data for the AI knowledge base.